# Model Routing Architecture for AI Coding Agent Harnesses

**First Author**[1], **Second Author**[2], **Third Author**[2], **Fourth Author**[1] **and Fifth Author**[1]
[*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

**AI coding agent harnesses operate in a market where model prices span three orders of magnitude (from $0.03 to $30 per million input tokens), yet capability gaps are sharply task-dependent: small models achieve 88–90% of frontier quality on single-function generation but catastrophically fail on complex multi-file tasks. Despite this, every major harness treats the model as a constant rather than a variable. We present a formal framework for *model routing architecture*: the problem of selecting, configuring, and composing multiple AI models across pipeline stages to optimize cost-quality-latency tradeoffs.**

**Our contributions are: (1) the *Heterogeneous Stage Assignment Problem*, which decomposes trivially in $O(MK)$ without capacity constraints, reduces to the Hungarian algorithm (Kuhn, 1955) under capacity constraints, and is conjectured NP-hard for the combined multi-objective case; (2) a *Cross-Model Diversity Result* applying Gaussian copula monotonicity to model routing, proving that same-family verification underestimates escape probability and that a weak independent verifier can outperform a strong correlated one; (3) the *Pipeline Crossover Theorem*, establishing the critical pipeline length $n^*$ beyond which a single expensive-model attempt dominates $K$ cheap-model attempts, with an implicit characterization as a function of per-step quality; (4) an *Adaptive Escalation Framework* grounding model escalation in sequential hypothesis testing (Wald's SPRT) and contextual bandits, with regret bounds of $O(d\sqrt{T \ln T})$ for the contextual setting and a switching-cost penalty of $\Theta(T^{2/3})$ when KV-cache invalidation is accounted for; and (5) a *Cascade Routing Architecture* inspired by Viola-Jones, where cheap models handle easy tasks and expensive models handle hard ones, achieving up to 85% cost reduction at 95% quality retention in published empirical evaluations.**

**We calibrate these formal results against benchmark data from SWE-bench, HumanEval, LiveCodeBench, and production routing systems (Cursor, RouteLLM, FrugalGPT), and identify routing collapse (the tendency of learned routers to default to the strongest model) as the primary failure mode requiring architectural mitigation. The framework provides a formal treatment of model selection specialized to multi-stage AI coding pipelines, connecting classical results from combinatorial optimization, copula theory, sequential testing, and online learning to the concrete engineering problem of multi-model harness design. We qualify this as the first formal treatment specialized to multi-stage coding agent pipelines; the general LLM routing and cascading literature (Dekoninck et al., 2025; Moslem and Kelleher, 2026) provides broader foundations that we draw on and extend.**

## 1. Introduction

The AI model market as of early 2026 presents a striking economic fact: input token prices range from $0.03 per million tokens (DeepSeek V3.2 with cache hits) to $30 per million tokens (Claude Opus 4.6 Fast Mode), spanning three orders of magnitude. Output token prices exhibit similar dispersion. Yet capability gaps are not proportional to price. On HumanEval (single-function generation), the cheapest models score 87–88% while frontier models score 97–98%, a gap of roughly 10 percentage points (Chen et al., 2021). On SWE-bench Verified (multi-file real-world bugs), frontier models cluster at 78–81% while budget models score 40–50%, and Haiku-class models effectively cannot perform the task at all.

This gap is *task-dependent*. For boilerplate code, type stubs, and well-specified single-function generation, small models capture 90%+ of frontier quality. For novel architectural decisions, complex multi-file refactors, and integration tasks requiring deep codebase understanding, frontier models are qualitatively better. No single model dominates across all task types at any price point.

Despite this, every major AI coding harness treats the model as a constant. GSD and Blueprint inherit the user's model choice. RAPID offers a configurable "model profile" (quality/speed) applied globally. Cursor implements the most sophisticated routing (different models for autocomplete vs. agent tasks), but without formal optimization. No existing harness formalizes model selection as an optimization problem, despite the magnitude of the opportunity: even a crude two-tier routing policy (cheap for simple, expensive for complex) can reduce costs by 50–80% while preserving 95%+ of quality (Ong et al., 2025).

This paper develops the formal framework for model routing architecture. We treat the model as a *variable* in the harness optimization problem, determining which model to use for each pipeline stage, how to configure it (temperature, tool access), and when to switch between models mid-pipeline. Our contributions are:

1. The **Heterogeneous Stage Assignment Problem** (§3): formalization of model-to-stage assignment under multiplicative quality, additive cost, and serial/parallel latency constraints. We show that unconstrained assignment decomposes trivially ($O(MK)$), that capacity-constrained assignment reduces to the Hungarian algorithm ($O(n^3)$), and that the multi-objective case is conjectured NP-hard but practically tractable via Lagrangian relaxation (Dekoninck et al., 2025).
2. The **Cross-Model Diversity Result** (§4): application of Gaussian copula monotonicity (Nelsen, 2006; Sklar, 1959) to model routing, proving that cross-family verification provides strictly positive diversity gain and establishing conditions under which a weak independent verifier outperforms a strong correlated one. This extends the qualitative FKG-based argument from the companion quality architecture paper into a quantitative copula-based framework.
3. The **Pipeline Crossover Theorem** (§5): derivation of the critical pipeline length $n^*$ beyond which a single expensive-model attempt dominates $K$ cheap-model attempts, connecting Pass@K and Pass$^K$ semantics.
4. An **Adaptive Escalation Framework** (§6): grounding model escalation in sequential hypothesis testing and contextual bandits, with formal regret bounds and switching-cost analysis for KV-cache invalidation.
5. A **Cascade Routing Architecture** (§7): synthesis of the formal results into a practical architecture with cheap-model-first processing, SPRT-based escalation, and budget-aware allocation.
6. **Empirical calibration** (§8) against published benchmarks and production systems, with honest engagement with routing collapse as the primary failure mode.

**Scope and framing.** This is a theory and position paper. Our novel contributions are in the *application* of established formal machinery (combinatorial optimization, copula theory, sequential testing, online learning) to the new problem of multi-model harness design, and in the *synthesis* of empirical data into calibrated formal models. The individual results draw on classical theorems; the contribution is their composition into a coherent framework and their calibration against the specific economics and performance characteristics of the 2025–2026 LLM market.

## 2. Preliminaries and Definitions

### 2.1. Notation

Let $\mathcal{M} = \{m_1, \ldots, m_M\}$ denote the set of available models, indexed by capability tier. Let $\mathcal{K} = \{1, \ldots, K\}$ denote the pipeline stages of an agent harness (e.g., planning, implementation, testing, review, integration). For each model $m \in \mathcal{M}$ and stage $k \in \mathcal{K}$:

- $q_k(m) \in [0, 1]$: success probability of model $m$ at stage $k$
- $c_k(m) > 0$: expected cost (tokens × price) of model $m$ at stage $k$
- $l_k(m) > 0$: expected latency of model $m$ at stage $k$
- $\rho(m_i, m_j) \in [0, 1]$: blind-spot correlation between models $m_i$ and $m_j$

**Definition 2.1** (Model Assignment)**.** *A* model assignment *is a function $\sigma : \mathcal{K} \to \mathcal{M}$ mapping each pipeline stage to a model. The space of all assignments is $\mathcal{M}^K$, with $|\mathcal{M}^K| = M^K$.*

**Definition 2.2** (Pipeline Quality)**.** *For assignment σ, the pipeline quality (end-to-end success probability under independence) is:*

$$Q(\sigma) = \prod_{k=1}^{K} q_k(\sigma(k)) \tag{1}$$

*This follows from the series reliability model: the pipeline succeeds only if every stage succeeds.*

**Definition 2.3** (Pipeline Cost and Latency)**.**

$$C(\sigma) = \sum_{k=1}^{K} c_k(\sigma(k)) \qquad \textit{(additive; always applies)} \tag{2}$$

$$L_{\text{serial}}(\sigma) = \sum_{k=1}^{K} l_k(\sigma(k)) \qquad \textit{(serial pipeline)} \tag{3}$$

$$L_{\text{parallel}}(\sigma) = \max_{k \in \mathcal{K}} l_k(\sigma(k)) \qquad \textit{(parallel stages)} \tag{4}$$

*Coding agent pipelines are typically serial (plan, then implement, then test), making $L_{\text{serial}}$ the default. $L_{\text{parallel}}$ applies when stages can execute concurrently (e.g., independent file edits).*

### 2.2. The Model Landscape

Table 1 summarizes the model landscape as of early 2026, calibrated from published benchmarks and API pricing.

Table 1 | Model landscape: price, quality, and latency by tier (early 2026).

| Tier | Representative | Input $/MTok | HumanEval | SWE-bench V. | TTFT (s) |
|---|---|---|---|---|---|
| Budget | DeepSeek V3.2 | $0.03–0.28 | 85–88% | 40–50% | 0.3–0.5 |
| Haiku-class | GPT-4o mini | $0.15 | 87% | <10% | 0.3–0.5 |
| Haiku-class | Claude Haiku 4.5 | $1.00 | 90–92% | 55–60% | 0.5–1.0 |
| Sonnet-class | Claude Sonnet 4.6 | $3.00 | 95–97% | 75–80% | 1.0–2.0 |
| Opus-class | Claude Opus 4.6 | $5.00 | 97–98% | 78–81% | 2.0–5.0 |
| Premium | Opus 4.6 Fast | $30.00 | 97–98% | 78–81% | 1.0–2.0 |

The 1000× price range between the cheapest and most expensive options, combined with task-dependent quality gaps ranging from 10 to 70+ percentage points, defines the optimization landscape for model routing.

## 3. The Heterogeneous Stage Assignment Problem

### 3.1. Single-Objective Tractability

Without capacity constraints (any number of stages may use the same model), the assignment $\sigma : \mathcal{K} \to \mathcal{M}$ decomposes into $K$ independent per-stage selections:

**Proposition 3.1** (Unconstrained Assignment Decomposes)**.** *When models have no capacity constraints, minimizing $C(\sigma) = \sum_k c_k(\sigma(k))$ subject to $Q(\sigma) \geq Q_{\min}$ decomposes into $K$ independent minimizations: for each stage $k$, select $\sigma(k) = \arg\min_{m \in \mathcal{F}_k} c_k(m)$, where $\mathcal{F}_k = \{m : q_k(m) \geq Q_{\min}^{1/K}\}$ is the feasible set. Total complexity is $O(MK)$.*

This decomposition is the common case for model routing: any model can serve any number of stages. The problem becomes non-trivial when capacity constraints bind (e.g., rate limits restricting how many concurrent stages can use a given model's API):

**Proposition 3.2** (Capacity-Constrained Assignment via Hungarian Algorithm)**.** *When each model $m$ can serve at most $b_m$ stages ($\sum_k 1[\sigma(k) = m] \leq b_m$), the problem becomes a generalized assignment problem. For the special case $b_m = 1$ (each model serves exactly one stage, requiring $M \geq K$), it reduces to the Linear Sum Assignment Problem (Kuhn, 1955; Munkres, 1957), solvable in $O(n^3)$ where $n = \max(M, K)$.*

**Proposition 3.3** (Multiplicative Objective Reduces to Additive)**.** *Maximizing $Q(\sigma) = \prod_k q_k(\sigma(k))$ subject to $C(\sigma) \leq C_{\max}$ reduces to an additive problem via $\tilde{q}_k(m) = \log q_k(m)$ (assuming all $q_k(m) > 0$; zero-probability model-stage pairs are excluded from the feasible set).*

### 3.2. Multi-Objective Hardness

**Conjecture 3.1** (Combined Assignment is Hard)**.** *The problem of simultaneously minimizing cost, maximizing quality, and bounding latency:*

$$\min_{\sigma} C(\sigma) \quad s.t. \quad Q(\sigma) \geq Q_{\min}, \;\; L(\sigma) \leq L_{\max} \tag{5}$$

*is NP-hard in general for the capacity-constrained case, by analogy to the multi-level bottleneck assignment problem. We state this as a conjecture rather than a theorem because the formal reduction requires constructing an instance mapping that we do not provide here. For the unconstrained case, the problem remains tractable via enumeration ($M^K$ is small for practical harness dimensions).*

### 3.3. Practical Tractability via Lagrangian Relaxation

For the model sizes relevant to AI coding harnesses ($M = 3$–$6$ model tiers, $K = 4$–$8$ pipeline stages), the assignment space is small: $M^K \in [81, 1{,}679{,}616]$. For $M = 3, K = 6$, exhaustive enumeration of all 729 assignments is trivial.

For larger instances, Dekoninck et al. (2025) provide a principled Lagrangian relaxation. Their key result:

**Proposition 3.4** (Lagrangian Scalarization (Dekoninck et al., 2025)). *The optimal routing strategy selects, for each task with features* x*, the model maximizing:*

$$\tau_i(\mathrm{x}, \lambda) = \hat{q}_i(\mathrm{x}) - \lambda \cdot \hat{c}_i(\mathrm{x}) \tag{6}$$

*where $\hat{q}_i$ and $\hat{c}_i$ are estimated quality and cost for model i, and $\lambda$ is a Lagrange multiplier controlling the cost-quality tradeoff.*

This reduces the multi-objective problem to a family of single-objective problems parameterized by $\lambda$. Sweeping $\lambda$ traces the Pareto frontier.

## 4. Cross-Model Diversity for Verification

### 4.1. Copula-Based Diversity Analysis

The qualitative argument for cross-model diversity originates in the FKG inequality (Fortuin et al., 1971): under a shared-difficulty model, producer and verifier failure probabilities are positively correlated (both increase with task difficulty), so independence *underestimates* escape probability. The companion quality architecture paper develops this via the Harris-FKG inequality on the difficulty lattice. Here we take a complementary quantitative approach using the Gaussian copula, which directly models the correlation parameter and yields numerical predictions.

**Definition 4.1** (Blind-Spot Correlation). *For models $m_i$ and $m_j$, define the latent Gaussian copula parameter $\rho(m_i, m_j)$ as the correlation of the underlying Gaussian variables in the copula representation of their joint failure distribution. This is related to, but not identical to, the Pearson correlation of binary failure indicators. Empirical estimates from Chen et al. (2026): same-family models have $\rho \approx 0.7$–$0.9$; cross-family models have $\rho \approx 0.3$–$0.5$.*

**Proposition 4.1** (Cross-Model Diversity Gain). *Let $m_P$ be the producer model and $m_V$ be the verifier model, with individual failure rates $\phi_P$ and $\phi_V$. The correlated escape probability (both miss a defect) under a Gaussian copula (Sklar, 1959) with parameter $\rho$ is:*

$$P_{\text{esc}}(\rho) = C_\rho(\phi_P, \phi_V) = \Phi_2(\Phi^{-1}(\phi_P), \Phi^{-1}(\phi_V); \rho) \tag{7}$$

*The diversity gain from using a cross-family verifier ($\rho_{\text{cross}}$) versus a same-family verifier ($\rho_{\text{same}}$) is:*

$$\Delta_{\text{div}} = C_{\rho_{\text{same}}}(\phi_P, \phi_V) - C_{\rho_{\text{cross}}}(\phi_P, \phi_V) > 0 \tag{8}$$

*This follows directly from the monotonicity of $\Phi_2$ in its correlation parameter (Nelsen, 2006).*

**Remark.** *We label this a proposition rather than a theorem because it is a direct consequence of known copula monotonicity, not a novel result. The contribution is the application to model routing and the quantitative calibration below.*

**Corollary 4.1** (Weak Independent Beats Strong Correlated). *A verifier with failure rate $\phi_V^{\text{weak}}$ and copula parameter $\rho_{\text{cross}} = 0.4$ (realistic cross-family) can achieve lower escape probability than a verifier with failure rate $\phi_V^{\text{strong}} < \phi_V^{\text{weak}}$ and $\rho_{\text{same}} = 0.8$ (same-family), provided $C_{0.4}(\phi_P, \phi_V^{\text{weak}}) < C_{0.8}(\phi_P, \phi_V^{\text{strong}})$.*

**Remark.** *Numerically: with $\phi_P = 0.15$, a cross-family verifier with $\phi_V = 0.40$ and $\rho = 0.4$ yields $P_{\text{esc}} = C_{0.4}(0.15, 0.40) \approx 0.08$. A same-family verifier with $\phi_V = 0.20$ and $\rho = 0.8$ yields $C_{0.8}(0.15, 0.20) \approx 0.11$. The weaker cross-family verifier achieves roughly 27% lower escape probability despite its lower individual detection rate.*

### 4.2. Empirical Calibration of Correlation

Ruis et al. (2025) measured error correlations across 350+ models on 20,000+ questions. Key findings relevant to routing:

- Within-family correlations (e.g., multiple GPT variants, or Claude Sonnet vs. Opus) range from 0.7 to 0.9.
- Cross-family correlations (e.g., Claude vs. GPT vs. Gemini) range from 0.3 to 0.5.
- More accurate models exhibit *more* correlated errors, not less, because they agree on which problems are hard.
- The ensemble error floor under uniform correlation $\rho$ is non-zero: increasing ensemble size cannot overcome structural correlations (Chen et al., 2026).

**Proposition 4.2** (Ensemble Saturation)**.** *Under a one-factor Gaussian copula with loading $\sqrt{\rho}$ (equicorrelated model), the probability that all $N$ models simultaneously fail converges to a non-zero floor as $N \to \infty$ whenever $\rho > 0$ (Chen et al., 2026). The precise floor depends on the copula structure, but the qualitative result is clear: adding more same-family models ($\rho \approx 0.7$–$0.9$) yields rapidly diminishing returns. Empirically, Chen et al. (2026) find that within-family ensembles saturate at $N = 3$–$4$, while cross-family ensembles ($\rho \approx 0.3$–$0.5$) continue to improve to larger $N$.*

### 4.3. Architectural Implication

The diversity theorem establishes a *hard constraint* for model routing: the verification model must be from a different model family than the production model. This means the harness must maintain API access to at least two model providers. The constraint is not an optimization variable; it is a structural requirement for effective quality assurance.

## 5. The Pipeline Crossover Theorem

### 5.1. Pass@$N$ versus Pass$^n$

Following the distinction introduced in Anthropic's evaluation methodology and formalized by Chen et al. (2021). We use $N$ for the number of sampling attempts and $n$ for pipeline length, to avoid overloading $K$ (which denotes the number of pipeline stages in §2).

**Definition 5.1** (Pass@$N$ and Pass$^n$)**.** *For a model with per-attempt success probability $q$:*

$$\text{Pass@}N = 1 - (1 - q)^N \qquad \textit{(at least one success in } N \textit{ attempts)} \tag{9}$$

$$\text{Pass}^n = q^n \qquad \textit{(all } n \textit{ sequential steps succeed)} \tag{10}$$

Pass@$N$ applies when the harness can select the best output from multiple attempts (e.g., code generation with test-based selection). Pass$^n$ applies when every step in a multi-step pipeline must succeed (e.g., plan, implement, test, review).

### 5.2. The Crossover Point

**Theorem 5.1** (Pipeline Crossover)**.** *Let $q_e$ be the per-step success probability of an expensive model and $q_c < q_e$ that of a cheap model, with cost ratio $r = c_e/c_c > 1$. For a pipeline of $n$ stages with Pass$^n$*

*semantics, a single expensive-model attempt dominates $N = r$ independent cheap-model attempts (at equal total cost) when:*

$$q_e^n > 1 - (1 - q_c^n)^r \tag{11}$$

*The crossover pipeline length $n^*$ is characterized implicitly by the equation $q_e^{n^*} = 1 - (1 - q_c^{n^*})^r$ and can be bracketed numerically. For $q_e = 0.85, q_c = 0.60, r = 5$: the crossover occurs between $n = 3$ and $n = 4$ (see Section A for numerical details).*

*Proof.* A single expensive attempt at $n$ stages succeeds with probability $q_e^n$. Running $N = r$ independent cheap attempts at $n$ stages, each succeeding with probability $q_c^n$, gives Pass@$N$ probability $1 - (1 - q_c^n)^N$. The expensive model dominates when $q_e^n > 1 - (1 - q_c^n)^N$. For the stated parameters ($N = r = 5$): at $n = 3$, $q_e^3 = 0.614$ versus cheap Pass@5 $= 1 - (1 - 0.216)^5 = 0.714$ (cheap wins). At $n = 4$: $q_e^4 = 0.522$ versus cheap Pass@5 $= 1 - (1 - 0.130)^5 = 0.504$ (expensive wins). The crossover occurs between $n = 3$ and $n = 4$. □

**Corollary 5.1** (Stage-Dependent Strategy)**.** *The optimal model selection strategy depends on pipeline structure:*

- *For single-stage or short pipelines ($n \leq n^*$): use cheap models with Pass@$N$ sampling.*
- *For long pipelines ($n > n^*$): use expensive models with Pass@1.*
- *For heterogeneous pipelines: assign models per-stage based on the local quality requirement.*

### 5.3. Budget Reallocation Evidence

Hassid et al. (2024) provide direct empirical confirmation: a 13B-parameter model generating 5 outputs outperformed a 70B model generating 1 output by up to 15% on HumanEval and MBPP under fixed compute budgets. However, this advantage vanishes when unit tests are unavailable for selection, confirming that Pass@$N$ requires a reliable verifier.

## 6. Adaptive Escalation Framework

### 6.1. The Escalation Decision

During pipeline execution, the harness may encounter tasks that exceed the assigned model's capability. The escalation policy decides when to switch to a more capable (and expensive) model.

**Definition 6.1** (Escalation Threshold)**.** *Escalate from cheap model $m_c$ to expensive model $m_e$ when the posterior probability that the cheap model's response is adequate falls below:*

$$p^* = \frac{c_e + s(C)}{c_e + s(C) + c_{\text{rework}}} \tag{12}$$

*where $c_e$ is the cost of the expensive model, $s(C)$ is the switching cost (KV-cache invalidation for context of length $C$), and $c_{\text{rework}}$ is the expected cost of fixing a failed cheap attempt.*

When the estimated probability of success with the cheap model drops below $p^*$, the expected cost of proceeding (failure cost × failure probability) exceeds the cost of escalating immediately.

### 6.2. Sequential Testing for Escalation

The escalation decision maps to Wald's Sequential Probability Ratio Test (Wald, 1945):

**Proposition 6.1** (SPRT-Based Escalation)**.** *Formulate two hypotheses:*

$$H_0 : \textit{cheap model response is adequate} \tag{13}$$

$$H_1 : \textit{cheap model response is inadequate} \tag{14}$$

*Accumulate evidence from quality signals (syntax validity, type-check pass, test results, self-evaluation score). The log-likelihood ratio* $\Lambda_k = \sum_{i=1}^{k} \log \frac{f_1(x_i)}{f_0(x_i)}$ *evolves with each signal. Accept* $H_0$ *(keep cheap response) when* $\Lambda_k \leq \log A$*; accept* $H_1$ *(escalate) when* $\Lambda_k \geq \log B$*, where* $A, B$ *are thresholds derived from the cost ratio.*

By the Wald-Wolfowitz optimality theorem (Wald and Wolfowitz, 1948), the SPRT minimizes the expected number of observations (quality signals) among all sequential tests achieving the same error rates. This means the SPRT-based escalation policy gathers the minimum amount of evidence before deciding.

### 6.3. Contextual Bandits for Task-Aware Routing

The model routing problem is fundamentally contextual: the best model depends on task features. We formalize this as a contextual bandit.

**Definition 6.2** (Model Routing as Contextual Bandit)**.** *At each task* $t$*:*

1. *The harness observes task features* $\mathrm{x}_t \in \mathbb{R}^d$ *(file complexity, task type, edit scope).*
2. *The harness selects model* $a_t \in \mathcal{M}$*.*
3. *The harness observes reward* $r_t(a_t) = q(a_t, \mathrm{x}_t) - \lambda \cdot c(a_t)$*.*

**Proposition 6.2** (Routing Regret Bounds (Known Results))**.** *The following established regret bounds apply to model routing:*

*(a)* ***Non-contextual*** *(fixed task distribution): UCB1 achieves* $O\left(\sum_{i:\Delta_i>0} \frac{\log T}{\Delta_i}\right)$ *regret, matching the Lai-Robbins lower bound (Auer et al., 2002; Lai and Robbins, 1985).*

*(b)* ***Contextual*** *(task-dependent routing): LinUCB achieves* $O(d\sqrt{T \log T})$ *regret for* $d$*-dimensional task features (Li et al., 2010).*

*(c)* ***General policy class****: The reduction to cost-sensitive classification achieves* $O(\sqrt{MT})$ *regret with oracle-efficient computation (Agarwal et al., 2014).*

### 6.4. Switching Costs and KV-Cache Invalidation

Model switching incurs a concrete cost: KV-cache invalidation. When the harness switches from model $m_A$ to model $m_B$ mid-conversation, the entire KV cache from $m_A$ is discarded. For a context of $C$ tokens, this forces recomputation costing $O(C)$ time and compute.

**Theorem 6.1** (Switching-Cost Regret Penalty (Dekel et al., 2014))**.** *With unit switching costs, the minimax regret for* $M$*-armed bandits over* $T$ *rounds is* $\Theta(M^{1/3}T^{2/3})$*, strictly worse than the* $O(\sqrt{MT})$ *rate without switching costs.*

**Corollary 6.1** (Batched Routing)**.** *By Perchet et al. (2016),* $O(\log \log T)$ *policy-update batches suffice to achieve near-optimal regret. In practice, re-evaluating the routing policy 3–5 times per day (rather than per-task) is theoretically near-optimal and eliminates unnecessary model switches.*

### 6.5. Budget-Constrained Routing

**Proposition 6.3** (Bandits with Knapsacks (Badanidiyuru et al., 2018)). *When routing is subject to a budget constraint $B$ (daily or monthly API cost limit), the Bandits with Knapsacks framework achieves regret optimal up to polylogarithmic factors relative to the optimal dynamic allocation policy.*

This provides the first principled framework for answering: "Given a $100/day API budget, how should I allocate tasks across model tiers to maximize aggregate quality?"

### 6.6. Sample Complexity

**Proposition 6.4** (Learning a Routing Policy). *The sample complexity for learning an $\epsilon$-optimal routing policy scales as:*

- *Non-contextual: $O(M/\Delta^2 \cdot \log(M/\delta))$ tasks. For $M = 3$ models, $\Delta = 0.1$: approximately 1,000 tasks.*
- *Contextual with $d$ features: $O(d^2/\epsilon^2)$ tasks. For $d = 10$, $\epsilon = 0.1$: approximately 10,000 tasks.*

Cold-start mitigation strategies include informative priors from benchmark data (Agrawal and Goyal, 2012), warm-start transfer from other codebases, and heuristic initialization (e.g., "use Sonnet by default, escalate to Opus on failure").

## 7. Cascade Routing Architecture

### 7.1. The Cascade Paradigm

Inspired by the Viola-Jones cascade (Viola and Jones, 2001, 2004), we propose a cascade routing architecture where tasks flow through models of increasing capability, with early exit when the current model's output is deemed adequate.

**Definition 7.1** (Model Cascade). *A* model cascade *is a sequence of models $m_1, m_2, \ldots, m_L$ ordered by capability and cost ($c_1 < c_2 < \cdots < c_L$, $q_1 \leq q_2 \leq \cdots \leq q_L$). For each task:*

1. *Query $m_1$. If the quality estimator scores the response above threshold $\tau_1$, return it.*
2. *Otherwise, query $m_2$ with the task (and optionally, $m_1$'s response as context). If above $\tau_2$, return.*
3. *Continue until $m_L$ (which always returns).*

**Theorem 7.1** (Cascade Expected Cost (Viola and Jones, 2004)). *For a cascade with $L$ stages where stage $i$ has acceptance probability $a_i$ (probability that the quality estimator accepts model $m_i$'s response), the expected cost per task is:*

$$\mathbb{E}[cost] = c_1 + (1 - a_1)c_2 + (1 - a_1)(1 - a_2)c_3 + \cdots = \sum_{i=1}^{L} c_i \prod_{j=1}^{i-1}(1 - a_j) \tag{15}$$

*When $a_1$ is large (most tasks are easy), the expected cost is dominated by $c_1$, the cheapest model.*

### 7.2. Quality Estimation: The Critical Factor

Dekoninck et al. (2025) identify quality estimation as the critical success factor for all model routing approaches. Good quality estimators are the sine qua non of effective routing; without them, all

routing strategies degrade to either (a) always selecting the expensive model (routing collapse) or (b) random selection.

Observable quality signals for coding tasks include:

- Syntax validity and type-check pass (binary, fast, deterministic)
- Test suite pass rate (high-signal but requires test execution)
- Model self-reported confidence (calibration varies by model)
- Retry count within a task (more retries signal harder tasks)
- Diff size and file count (proxy for task complexity)

### 7.3. Routing Collapse and Mitigation

**Definition 7.2** (Routing Collapse (Zhang et al., 2026))**.** Routing collapse *occurs when a learned router systematically defaults to the most capable (and expensive) model, even when cheaper models would suffice. On RouterBench, existing routers send approximately 100% of queries to the strongest model under loose budgets, while the oracle optimal strategy uses it for fewer than 20%.*

The root cause is an *objective-decision mismatch*: routers are trained on scalar quality prediction but must make discrete ranking decisions at deployment. When 94.9% of queries have near-identical predicted quality across models (the "small-margin regime"), minimal perturbations in the quality estimator flip routing decisions.

**Proposition 7.1** (Mitigation via Cost-Penalized Routing)**.** *Routing collapse is mitigated by replacing quality-maximizing routing with cost-penalized routing (the Lagrangian form of Equation (6)). By explicitly penalizing cost in the routing objective, the router is incentivized to select cheaper models whenever quality differences are within the noise margin.*

Additional mitigations include:

- **EquiRouter** (rank-based supervision): directly supervise per-query model rankings rather than scalar quality scores, reducing the objective-decision mismatch.
- **Cascading over routing**: instead of predicting which model is best *a priori*, try the cheap model first and escalate only on evidence of inadequacy. This converts the hard pre-routing problem into an easier post-hoc quality assessment.
- **Budget enforcement**: hard budget caps force the router to reserve expensive-model calls for genuinely hard tasks.

## 8. Empirical Calibration

### 8.1. Cost Savings from Routing

Table 2 summarizes published cost savings from model routing systems.

### 8.2. Task-Dependent Capability Gaps

Table 3 quantifies the capability gap by task type, confirming that routing value concentrates in the extremes: very easy tasks (where cheap models suffice) and very hard tasks (where only frontier models succeed).

Table 2 | Published cost savings from model routing systems.

| System | Cost Reduction | Quality Retained | Source |
|---|---|---|---|
| RouteLLM (MT Bench) | 85% | 95% of GPT-4 | Ong et al. (2025) |
| RouteLLM (MMLU) | 45% | 92% | Ong et al. (2025) |
| FrugalGPT | up to 98% | matches GPT-4 | Chen et al. (2024) |
| MetaLLM | up to 60% | +1% accuracy | Nguyen et al. (2024) |
| Cascade routing | 14% perf. gain | same cost | Dekoninck et al. (2025) |
| Budget reallocation | 75% compute | +15% HumanEval | Hassid et al. (2024) |

Table 3 | Capability gap by task type (early 2026 benchmarks).

| Task Type | Budget Model | Frontier Model | Gap |
|---|---|---|---|
| Single-function (HumanEval) | 87–88% | 97–98% | ~10 pp |
| Multi-file bugs (SWE-bench V.) | 40–50% | 78–81% | ~35 pp |
| Complex editing (Aider polyglot) | 70% ($0.88) | 88% ($29) | 18 pp, 33× cost |
| Competitive programming (LCB) | 79% | 92% | ~13 pp |
| Merge conflict resolution | LLaMA-8B best | CodeLlama-34B worse | Negative |

The merge conflict result (Zhu et al., 2024) is particularly striking: general-purpose models outperform code-specialized models on a code task, suggesting that routing should consider model *type*, not just model *size*.

### 8.3. Production Routing Architectures

Cursor processes over 400 million predictions per day with task-specific routing: custom fine-tuned models for autocomplete (sub-second latency), GPT variants for background summarization, and frontier models for agent/multi-file tasks. Their Tab RL system uses online reinforcement learning to optimize suggestion quality, deploying new models several times per day with 1.5–2 hour training loops.

Claude Code provides user-directed routing with an "OpusPlan" strategy: automatically switch to Opus 4.6 for architecture and planning, then to Sonnet 4.6 for code generation. This is a two-tier cascade that recognizes the task-dependent capability gap.

GitHub Copilot's "Smart Mode" routes each request to the most appropriate model variant or reasoning depth, drawing from a pool of 9+ models across three providers.

## 9. Related Work

**LLM routing.** Moslem and Kelleher (2026) provide the first comprehensive survey of dynamic model routing, identifying six paradigms: difficulty-aware, preference-aligned, clustering-based, reinforcement learning, uncertainty quantification, and cascading. Our framework provides the formal foundations that this survey identifies as missing. Ong et al. (2025) demonstrate empirical routing effectiveness but lack formal optimality guarantees. Dekoninck et al. (2025) provide the first optimality proofs for LLM routing and cascading, which we extend with the assignment-theoretic and diversity-theoretic results.

**Mixture of experts.** The intra-model routing in Mixture-of-Experts architectures (Fedus et al., 2022; Shazeer et al., 2017) selects which expert sub-network processes each token. Our inter-model routing operates at a coarser granularity (selecting which complete model processes each task) but shares the fundamental principle: route inputs to the most appropriate processing unit.

**Cascade classifiers.** The Viola-Jones cascade (Viola and Jones, 2001, 2004) and its extensions (Saberian and Vasconcelos, 2014) established the cheap-rejection paradigm. FrugalGPT (Chen et al., 2024) first applied this to LLM inference. Our contribution is connecting the cascade paradigm to the formal bandit and sequential testing literature.

**Multi-agent verification.** Lifshitz et al. (2025) show that diverse verifier ensembles outperform homogeneous ones, empirically confirming our Cross-Model Diversity Theorem. Ruis et al. (2025) provide the correlation data we use for calibration.

**Online learning.** The bandit framework (Auer et al., 2002; Lattimore and Szepesvári, 2020) and its contextual extension (Agarwal et al., 2014; Li et al., 2010) provide the regret-theoretic foundations. The switching-cost analysis (Dekel et al., 2014) and batched bandits (Perchet et al., 2016) address the KV-cache invalidation problem. The Bandits with Knapsacks framework (Badanidiyuru et al., 2018) handles budget constraints.

**Learning to defer.** Mozannar and Sontag (2020) provide consistent surrogate losses for the deferral decision, directly applicable to the escalation classifier in our cascade.

## 10. Contrarian Positions

We engage five objections to formal model routing, presenting each in its strongest form before responding.

**1. "Single-model simplicity wins."** Gabriel's "worse is better" principle (Gabriel, 1989) and Fowler's microservice premium argue that implementation simplicity dominates interface correctness. Model routing is the LLM equivalent of premature microservice decomposition: it splits a single-model "monolith" into a distributed system of specialized model calls, inheriting all coordination overhead. Prompt format incompatibility across model families (XML for Claude, Markdown for GPT, priority ordering for Gemini) multiplies maintenance cost.

*Response.* The argument is strong for small-scale usage. For organizations processing fewer than 10,000 tokens per day, routing overhead likely exceeds savings. But the 1000× price range is not ignorable at scale. RouteLLM achieves 85% cost reduction with minimal architectural complexity by using a lightweight router (a fine-tuned BERT classifier) rather than a distributed multi-model system. The relevant comparison is not "monolith vs. microservices" but "monolith vs. monolith with a routing header." Abstraction layers (OpenRouter, LiteLLM) further reduce integration burden.

**2. "Frontier models will be cheap enough."** LLM inference costs have declined approximately 1000× in three years. If this trajectory continues, routing becomes unnecessary as frontier quality approaches commodity pricing.

*Response.* The headline decline is real, but masks a K-shaped bifurcation: commodity inference is racing to zero while frontier reasoning models are getting *more* expensive. Google's Flash model surged 6.7× on input pricing between August 2024 and December 2025. The ratio between model tiers may be widening, not narrowing. Additionally, Jevons paradox predicts that cheaper inference induces larger context windows, deeper reasoning chains, and more agent steps, maintaining cost pressure. Price elasticities near unity confirm that demand scales with price reductions.

**3. "Cross-model verification is too complex."** Each additional model API is a dependency, failure point, and latency source. LLM API reliability (~99.76% uptime) is dramatically worse than mature SaaS (~99.99%), and multi-provider dependencies compound outage risk.

*Response.* The reliability concern is valid but partly self-defeating: multi-provider architectures can improve reliability through failover. The strongest form of this objection targets synchronous, blocking cross-model verification in the critical path. Asynchronous verification (check after the fact, flag rather than block) decouples verification latency from user-facing latency.

**4. "Best-of-N sampling wastes resources."** The Inference Scaling FLaws paper (Beeching et al., 2024) shows that optimal $K \leq 5$ for resampling with imperfect verifiers, and the Gaussian-copula ensemble model (Chen et al., 2026) establishes a hard saturation floor on ensemble gains under correlation.

*Response.* These are genuine limits on *naive* Best-of-N. Adaptive compute allocation (spending more samples on hard tasks, fewer on easy ones) improves efficiency 4× over uniform sampling. Cross-family diversity ($\rho \approx 0.4$) provides meaningful improvement over within-family sampling ($\rho \approx 0.8$). The routing framework converts the static sampling problem into a dynamic allocation problem, partially circumventing the saturation floor.

**5. "Task difficulty prediction is too noisy."** The routing collapse finding (Zhang et al., 2026), where 94.9% of queries occupy a small-margin regime making routing decisions essentially random, is the most damaging empirical evidence against routing.

*Response.* We take this objection seriously. Routing collapse is real and documented. However, the aggregate cost savings from RouteLLM (85% reduction at 95% quality) demonstrate that routing works at the portfolio level even when individual decisions are noisy. The cascade architecture partially sidesteps the prediction problem: instead of predicting difficulty *a priori*, try the cheap model first and escalate only on evidence of failure. This converts a hard classification problem ("is this task easy or hard?") into an easier verification problem ("is this response good enough?").

## 11. Design Principles

The formal results yield six design principles for model routing in AI coding harnesses:

1. **Cascade, don't predict.** The cascade architecture (cheap model first, escalate on failure) is more robust than pre-routing (predict difficulty, select model). Pre-routing requires accurate difficulty estimation; cascading requires only adequate quality assessment, which is easier.
2. **Cross-family verification is a hard constraint.** The FKG inequality makes same-family verification systematically biased. The verifier must come from a different model family than the producer. This is not negotiable.

3. **Batch routing decisions.** KV-cache invalidation costs make per-task model switching expensive. Commit to a model for an entire conversation or task phase, re-evaluating only at natural breakpoints. Three to five policy updates per day is near-optimal (Corollary 6.1).
4. **Budget-aware allocation.** Use the Bandits with Knapsacks framework to allocate tasks under API cost budgets. This naturally balances quality and cost without manual threshold tuning.
5. **Use pipeline length to select strategy.** For short pipelines ($n \leq 3$), use cheap models with Pass@$N$ sampling. For long pipelines ($n > 3$), invest in per-step quality with expensive models.
6. **Monitor for routing collapse.** Track the fraction of tasks routed to the most expensive model. If it exceeds 80%, the router has likely collapsed and should be recalibrated or replaced with a cascade.

## 12. Limitations

1. **Benchmark calibration, not experiments.** All empirical data comes from published benchmarks and production reports, not controlled experiments. The specific numbers (85% cost reduction, 1000× price range) reflect a snapshot of the rapidly evolving 2025–2026 market and will change.
2. **Independence assumptions.** The multiplicative quality model ($Q = \prod q_k$) assumes stage-level independence. In practice, errors propagate: a bad plan increases implementation failure probability. The compound error framework from the reliability architecture paper in this series provides corrections, but we do not integrate them here.
3. **Quality estimation gap.** The cascade architecture's effectiveness depends entirely on quality estimation accuracy. We identify this as the critical factor but do not solve it. Current quality estimators (test pass rate, self-evaluation, syntax checks) are noisy and domain-specific.
4. **Non-stationarity.** Model capabilities change with provider updates, sometimes dramatically. The bandit framework assumes stationarity or slow drift. Handling abrupt model updates (e.g., a provider deprecating a model tier) requires mechanisms beyond the current framework.
5. **Multi-step dependencies.** Coding tasks span multiple model invocations where the routing decision for step $t$ affects context for step $t+1$. This creates a Markov Decision Process, not just a bandit. The MDP formulation is an open problem for future work.
6. **Routing collapse is unsolved.** We identify routing collapse as the primary failure mode and propose mitigations (cascading, cost penalization, rank-based training), but do not prove that any mitigation eliminates it.

## 13. Conclusion

Model routing is the single largest untapped optimization in AI coding agent harness design. The economics are stark: a 1000× price range across models, task-dependent capability gaps ranging from 10 to 70+ percentage points, and published evidence of 85% cost reduction at 95% quality retention. Yet no major harness formalizes model selection as an optimization problem.

We have provided a formal framework grounded in classical results: the Hungarian algorithm for stage assignment, the FKG inequality and Gaussian copula for diversity analysis, Wald's SPRT for escalation, and contextual bandits for adaptive routing. The key architectural insight is that cascading (try cheap, escalate on failure) is strictly more robust than routing (predict difficulty, select model), because cascading replaces a hard classification problem with an easier verification problem.

The framework has clear limitations. Quality estimation remains the bottleneck. Routing collapse is a real and documented failure mode. Non-stationarity and multi-step dependencies require MDP-level formalization that exceeds the bandit framework. But the opportunity is large enough, and the

formal tools mature enough, that model routing should be a first-class architectural concern in every AI coding harness, not an afterthought.

## References

A. Agarwal, D. Hsu, S. Kale, J. Langford, L. Li, and R. E. Schapire. Taming the monster: A fast and simple algorithm for contextual bandits. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 1638–1646, 2014.

S. Agrawal and N. Goyal. Analysis of Thompson Sampling for the multi-armed bandit problem. In *Conference on Learning Theory (COLT)*, pages 39.1–39.26, 2012.

P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

A. Badanidiyuru, R. Kleinberg, and A. Slivkins. Bandits with knapsacks. In *Journal of the ACM*, volume 65, pages 1–55, 2018. Originally appeared at FOCS 2013.

E. Beeching, C. Fourrier, N. Habib, S. Han, N. Lambert, N. Rajani, O. Sanseviero, L. Tunstall, and T. Wolf. Inference scaling FLaws: The limits of LLM resampling with imperfect verifiers. *arXiv preprint arXiv:2411.17501*, 2024. Optimal K ≤ 5; saturation under imperfect verifiers.

G. Chen, M. Liao, J. Li, and K. Fan. Don't always pick the highest-performing model: Error correlation in ensembles. *arXiv preprint arXiv:2602.08003*, 2026. Gaussian-copula model; within-family correlations 0.7–0.8.

L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv preprint arXiv:2305.05176, 2023.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

O. Dekel, J. Ding, T. Koren, and Y. Peres. Bandits with switching costs: $t^{2/3}$ regret. *arXiv preprint arXiv:1310.2997*, 2014. STOC 2014.

J. Dekoninck, M. Baader, and M. Vechev. A unified approach to routing and cascading for LLMs. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv preprint arXiv:2410.10347.

W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

C. M. Fortuin, P. W. Kasteleyn, and J. Ginibre. Correlation inequalities on some partially ordered sets. *Communications in Mathematical Physics*, 22(2):89–103, 1971.

R. P. Gabriel. The rise of "worse is better". 1989. Published in Lisp: Good News, Bad News, How to Win Big.

M. Hassid, T. Lacroix, T. Bhatt, G. Synnaeve, A. Szlam, and E. Grave. The larger the better? improved LLM code-generation via budget reallocation. *arXiv preprint arXiv:2404.00725*, 2024. ICLR 2025.

H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

T. L. Lai and H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.

T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.

L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670, 2010.

O. Lifshitz et al. Multi-agent verification for zero-shot code generation. *arXiv preprint*, 2025.

Y. Moslem and J. D. Kelleher. Dynamic model routing and cascading for efficient LLM inference: A survey. *arXiv preprint arXiv:2603.04445*, 2026.

H. Mozannar and D. Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 7076–7087, 2020.

J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.

R. B. Nelsen. *An Introduction to Copulas*. Springer, 2 edition, 2006.

Q. H. Nguyen et al. MetaLLM: A high-performant and cost-efficient dynamic framework for wrapping LLMs. *arXiv preprint arXiv:2407.10834*, 2024.

I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. arXiv preprint arXiv:2406.18665.

V. Perchet, P. Rigollet, S. Chassang, and E. Snowberg. Batched bandit problems. *The Annals of Statistics*, 44(2):660–681, 2016.

L. Ruis et al. Correlated errors in large language models. *arXiv preprint arXiv:2506.07962*, 2025. 350+ models, 20,000+ questions; more accurate models have more correlated errors.

M. J. Saberian and N. Vasconcelos. Boosting algorithms for detector cascade learning. *Journal of Machine Learning Research*, 15:2569–2605, 2014.

N. Shazeer, A. Mirhoseini, K. Mahdavi, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017. ICLR 2017.

A. Sklar. Fonctions de répartition à $n$ dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.

P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 511–518, 2001.

P. Viola and M. J. Jones. Robust real-time face detection. *International Journal of Computer Vision*, 57(2):137–154, 2004.

A. Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2): 117–186, 1945.

A. Wald and J. Wolfowitz. Optimum character of the sequential probability ratio test. volume 19, pages 326–339, 1948.

Y. Zhang, Z. Feng, and Z. Zhou. When routing collapses: Understanding systematic failures in LLM model selection. *arXiv preprint arXiv:2602.03478*, 2026. 94.9% of queries in small-margin regime; routers default to strongest model.

Y. Zhu et al. ConGra: Context-grounded merge conflict resolution. *arXiv preprint*, 2024.

# Appendices

## A. Proof Details

### A.1. Gaussian Copula Monotonicity in $\rho$

The Gaussian copula $C_\rho(u, v) = \Phi_2(\Phi^{-1}(u), \Phi^{-1}(v); \rho)$ is strictly increasing in $\rho$ for $u, v \in (0, 1)$. This follows from the representation $\Phi_2(x, y; \rho) = \int_{-\infty}^{x} \Phi\left(\frac{y - \rho t}{\sqrt{1-\rho^2}}\right) \phi(t)\, dt$, where $\frac{\partial}{\partial \rho}\Phi_2 = \phi_2(x, y; \rho) > 0$ for all finite $x, y$ (Nelsen, 2006).

### A.2. Pipeline Crossover Numerical Examples

For $q_e = 0.85$, $q_c = 0.60$, $r = c_e/c_c = 5$:

| $n$ | $q_e^n$ | $q_c^n$ | Pass@5 cheap | Winner |
|---|---|---|---|---|
| 1 | 0.850 | 0.600 | 0.990 | Cheap |
| 2 | 0.722 | 0.360 | 0.893 | Cheap |
| 3 | 0.614 | 0.216 | 0.714 | Cheap |
| 4 | 0.522 | 0.130 | 0.504 | Expensive |
| 5 | 0.444 | 0.078 | 0.337 | Expensive |

Table 4 | Pipeline crossover: expensive model dominates beyond $n = 3$ stages.

### A.3. Regret Bound Comparison

### A.4. Empirical Correlation Data

From Chen et al. (2026), measured on MEDMCQA:

## B. Existing Harness Model Selection

| Setting | Regret | Reference |
|---|---|---|
| Non-contextual (UCB1) | $O\left(\frac{M \log T}{\Delta}\right)$ | Auer et al. (2002) |
| Contextual (LinUCB) | $O(d\sqrt{T \log T})$ | Li et al. (2010) |
| General policy (Monster) | $O(\sqrt{MT})$ | Agarwal et al. (2014) |
| With switching costs | $\Theta(M^{1/3}T^{2/3})$ | Dekel et al. (2014) |
| Budget-constrained (BwK) | Optimal to polylog | Badanidiyuru et al. (2018) |

Table 5 | Regret bounds for model routing under different settings.

| Model Pair Type | Correlation $\rho$ |
|---|---|
| Same-family (e.g., GPT-4o, GPT-4 Turbo) | 0.7–0.9 |
| Cross-family (e.g., Claude vs. GPT) | 0.3–0.5 |
| Cross-modality (e.g., Claude vs. open-source) | 0.2–0.4 |

Table 6 | Empirical blind-spot correlation by model pair type.

| Harness | Model Selection | Cross-Model | Adaptive |
|---|---|---|---|
| GSD | User's model; model_profile | No | No |
| RAPID | Configurable per project | No | No |
| Turing | Researcher vs. evaluator (same family) | No | No |
| Blueprint | Single model; persona uniform | No | No |
| Cursor | Task-type routing (autocomplete vs. agent) | No | Partially |
| Claude Code | User-directed; OpusPlan | No | Partially |
| GitHub Copilot | Smart Mode (multi-model pool) | No | Partially |

Table 7 | Model selection approaches in existing AI coding harnesses.